

```
<script type="text/javascript">
  var x = new ActiveXObject("Microsoft.XMLHTTP");
  // var x = new XMLHttpRequest();
  x.open("TRACE", "http://example.com", false);
  x.setRequestHeader("Max-Forwards", "0");
  x.send();
  //x.send("");
  c=x.responseText;
  alert(c);
</script>
```

(Again, the base code is for Internet Explorer; modifications for Mozilla are commented out.) Microsoft, trying to secure Internet Explorer, removed the support for any method starting with *TRACE* in the *XmlHttp* object. However, this security measure was also broken when it was found that instead of using “*TRACE*”, the attacker can simply use “*\r\nTRACE*”. Hence, the line in the above script, `x.open(“TRACE”, “http://example.com”, false)` would become `x.open(“\r\nTRACE”, “http://example.com”, false)`.

Cross-site tracing (XST) is one of the most prevalent threats on the Internet today. However, following the security tips below can help you curb it.

Time for security

1. The first and the foremost security step to take is to disable the *TRACE* request method (unless needed) on your Web servers.
2. Moreover, Web server vendors should have *TRACE* disabled in the Web server’s default out-of-the-box configuration.
3. Proxy servers should also be shipped with *TRACE* disabled in their default configurations.
4. Disable *TRACE* in your browser’s *XmlHttpRequest* object too. For this, check the support page of your browser’s vendor.
5. For all Internet Explorer users: if you must continue using Internet Explorer, switch to version 7 or 8, because they seem to have patches for this attack. However, the far better option is for you to use the latest versions of Mozilla Firefox (>3.2) instead.

How to disable *TRACE* in Apache

Disabling *TRACE* in Apache is quite easy. The only thing one requires is the Apache *mod_rewrite* module installed, and then follow the steps shown below:

1. Activate *mod_rewrite* in *httpd.conf* by adding the following line to it:

```
LoadModule rewrite_module modules/mod_rewrite.so
```

2. Add the following lines in *httpd.conf* to disable *TRACE*:

```
RewriteEngine On
```

```
RewriteCond %{REQUEST_METHOD} ^TRACE
RewriteRule .* - [F]
```

3. Now, restart the Apache Web server.

After *TRACE* has been disabled according to the instructions above, any incoming *TRACE* requests will be responded to with an HTTP status code of either 403 or 405. You can also verify it by telneting to your server, as shown below:

```
server@attacker-$ telnet www.victim.com 88
OPTIONS / HTTP/1.1
Host: www.victim.com
HTTP/1.1 200 OK
Server: Apache-httpd/2.2.1
Date: Tue, 31 Oct 2006 08:00:29 GMT
Connection: close
Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS
Content-Length: 0
```

As we can see in the example, the *Allow* line provides a list of the HTTP methods that are supported by the Web server. In this case, we see that every method is enabled except for the *TRACE* method.

Let’s now move on to another Web application attack: cross-site history manipulation (XSHM)

Cross-site history manipulation (XSHM)

This recently evolved attack works by taking advantage of an individual’s browsing history, seen in Mozilla Firefox, Google Chrome and Internet Explorer. By manipulating the browser history, it is possible to compromise a Web browser’s same-origin policy (SOP), and so violate user privacy.

Recall that in Part 3 of this series, we discussed a bit about SOP, and also that Web pages from different origins cannot communicate with each other (‘communicate’ here means that a page from one origin can only send an HTTP request to a page from different origin; however, it cannot read an HTTP response of a page from a different origin). Hence, during CSRF (Cross Site Request Forgery), attackers can only submit malicious HTTP requests to a bank website, but they cannot read the HTTP response from that site. However, recently it was found that even these limitations for the attacker can be bypassed by compromising SOP, via manipulation of the browser’s history objects. First, let’s look at the design of the browser history object.

Browser history object

Browser history is a global list of pages visited by the user, which the user can cycle through by pressing the *Back* and *Forward* buttons of the browser. Some of its features are:

1. If the same URL is opened multiple times, only one entry will be made into the history list.
2. If a user opens Page B, which is then automatically redirected to Page A (by the Web server), then only the URL of Page A will be entered into the history list.